

Evidently AI Monitoring Report

1. Executive summary

This project treats cerebrovascular-accident prediction as a **binary classification** problem. The labelled `Known_Patients_01.csv` dataset is used to prepare the data, compare several classifiers, select the strongest baseline model, and generate predictions for `New_Patients_01.csv`. The current implementation selects the model with the highest holdout F1 score. In the tested workflow, this was Logistic Regression.

Evidently AI is used after model selection to monitor whether the new-patient data and the model's outputs are different from the reference population. It produces three HTML reports: a **data-drift report**, a **prediction-drift report**, and a **model-performance report**. These reports support model monitoring and investigation, but they do not replace clinical validation or medical decision-making.

2. Datasets and objective

Each row represents one patient. The features include age, hypertension, heart disease, glucose level, body mass index, gender, work type, residence type, marital status, and smoking status. The target column is `cerebrovascular_accident`, where `0` indicates no recorded cerebrovascular accident and `1` indicates a recorded cerebrovascular accident.

The known-patient dataset contains both features and target labels. It is therefore suitable for supervised learning. The new-patient dataset contains the same input fields, but its target values are unavailable. Its purpose is to receive predictions and to act as the current dataset for monitoring.

Dataset	Role	Target available?
<code>Known_Patients_01.csv</code>	Reference and labelled modelling data	Yes
<code>New_Patients_01.csv</code>	Current/new-patient monitoring data	No

The operational objective is to identify patients who may be at higher risk and may require further clinical assessment. A model prediction is a screening aid and must not be interpreted as a diagnosis.

3. Data preparation

The pipeline first separates the target from the input features. The identifier column is excluded because an anonymised patient ID is not a meaningful clinical predictor and could introduce spurious patterns.

Numeric columns are imputed with their median values. Median imputation is robust to extreme values and keeps the transformation deterministic. Categorical columns are imputed with their most frequent value. Categorical variables are then converted using one-hot encoding so that the classifiers can process them numerically. Numeric variables are standardised where required by the model pipeline.

The preprocessing transformer is fitted using the known patients only. It is then applied to both the known and new patients. This prevents information from the current/new dataset from influencing the preparation of the reference data and reduces the risk of data leakage.

4. Model training and selection

The target has two classes, so the task is classification rather than continuous-value regression. The project compares Logistic Regression, Decision Tree, K-Nearest Neighbours, Naive Bayes, Support Vector Machine, and Random Forest classifiers. The advanced workflow can additionally evaluate AdaBoost, XGBoost, SMOTE-based strategies, Optuna tuning, and LightGBM when those packages are installed.

The baseline evaluation uses a stratified holdout split. Stratification preserves the class proportions in both partitions, which is important because cerebrovascular-accident cases are a minority class. The workflow also uses cross-validation on the training partition.

Accuracy alone is not a sufficient selection criterion for this medical screening problem. A model can achieve high accuracy by predicting the majority class while missing many positive cases. The workflow therefore reports precision, recall, F1 score, and ROC AUC. The current selection rule ranks the baseline models by holdout F1 score because F1 balances precision and recall.

Metric	Meaning in this project
Accuracy	Proportion of all predictions that are correct. It can be misleading under class imbalance.
Precision	Proportion of predicted positive patients who are actually positive.
Recall	Proportion of actual positive patients detected by the model.
F1 score	Harmonic mean of precision and recall.

ROC AUC

Ability to rank positive cases above negative cases across probability thresholds.

The probability threshold affects the precision–recall balance. A lower threshold generally identifies more possible positive cases, increasing recall while possibly reducing precision. The threshold should be selected using the clinical cost of missed cases and false alarms, not chosen only to maximise accuracy.

5. What Evidently AI does

[Evidently AI](#) is an open-source tool for evaluating and monitoring machine-learning systems. It compares reference and current datasets, calculates statistical and model-quality metrics, and presents the results in interactive reports.

In this project, the known patients are the **reference** dataset and the new patients are the **current** dataset. This follows Evidently's monitoring model: the reference describes the population on which the model was developed, while the current data represents the population now entering the model.

5.1 Data drift

The data-drift report compares the distributions of the input features in the known and new datasets. For example, it can reveal changes in the distributions of age, glucose level, BMI, smoking status, or hypertension status.

A detected drift does not automatically mean that the model is wrong. It means that the current input population differs statistically from the reference population. Large or clinically meaningful drift can indicate a change in the patient population, data collection process, missing-value pattern, or hospital operating environment. It should trigger investigation and possibly additional validation or retraining.

The report is saved as:

Plain Text

```
reports/evidently/data_drift.html
```

5.2 Prediction drift

The prediction-drift report compares the selected model's outputs for known patients with its outputs for new patients. It monitors both the predicted class and the predicted probability.

Prediction drift is useful when the current patients do not yet have confirmed outcomes. It can show that the model is producing a substantially different mixture of positive and negative predictions, or that its confidence distribution has changed. However, prediction drift is only a proxy for model-quality drift. It cannot establish whether the predictions are correct because the new outcomes are not available.

The report is saved as:

Plain Text

```
reports/evidently/prediction_drift.html
```

5.3 Model-performance monitoring

The model-performance report evaluates classification quality using labelled data. Because the new-patient file has no observed target values, the report uses a leakage-safe comparison between a labelled training partition and a labelled stratified holdout partition from the known dataset.

This report includes classification information such as accuracy, precision, recall, F1 score, ROC AUC, confusion-matrix information, and probability-based diagnostics where supported by the installed Evidently version. It provides evidence about model generalisation, but it is not a measurement of the model's future performance on the new patients.

Actual current-cohort model performance can be calculated only after the new patients receive confirmed outcome labels. At that point, the labelled current dataset should be passed through the classification monitoring workflow and compared with the reference performance.

The report is saved as:

Plain Text

```
reports/evidently/model_performance_drift.html
```

6. How to run the reporting script

From the project directory, activate the virtual environment and install the dependencies:

Bash

```
cd /home/kingcole/Documents/uniAssignments/year_3_sem_1/ai_week_1/Assignment-02
source .venv/bin/activate
```

```
pip install -r requirements.txt
```

Run the standalone reporting script:

Bash

```
python evidently_report.py
```

The script loads the two CSV files, compares the candidate classifiers, selects the best baseline model by holdout F1 score, and generates the Evidently reports. The default output directory is:

Plain Text

```
reports/evidently/
```

The output files are:

File	Purpose
data_drift.html	Compares feature distributions between known and new patients.
prediction_drift.html	Compares predicted classes and probabilities between the two populations.
model_performance_drift.html	Evaluates labelled train/holdout classification performance.
summary.json	Stores a machine-readable summary of the run and report paths.

Custom input and output locations can be supplied as follows:

Bash

```
python evidently_report.py \  
  --known Known_Patients_01.csv \  
  --new New_Patients_01.csv \  
  --output reports/evidently
```

An HTML report can be opened from the terminal with:

Bash

7. Interpretation and limitations

The most important interpretation is whether the new-patient population remains similar enough to the reference population for the model's predictions to be considered familiar. Drifted columns should be investigated using clinical knowledge and data-quality checks. A statistical drift result alone does not prove a clinically important change.

The model should not be deployed as an autonomous diagnostic system. The data is anonymised, the target may reflect historical recording practices, and the new-patient labels are not yet available. The model requires external validation, threshold review, monitoring of false negatives and false positives, and appropriate clinical governance before it can support real patient-care decisions.

Evidently provides monitoring evidence rather than a guarantee of safety. Its reports help answer three questions: **Has the input population changed? Has the model's output distribution changed? Does the model still perform acceptably when labels are available?**

References

- [1] Evidently AI documentation: Introduction and Reports
- [2] Evidently AI documentation: Data Drift Preset
- [3] Evidently AI documentation: Classification Preset
- [4] scikit-learn documentation: Model Evaluation